

ar Guía de Actividades Práctico-Experimentales Nro. 001

ar Grupo H

Integrantes: Valeria Agila, José Valencia, Domenica Narváez, Sherman Abarca, Diyer Torres, Gabriel Suarez

- **Configuración del Entorno**

Importar librerías necesarias:

```
import itertools
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# Verificar importaciones
print("Librerías importadas correctamente")
```

Librerías importadas correctamente

ar Tarea 1: Generación de Espacios Muestrales (Ω)

1. Genere el espacio muestral para el lanzamiento de una moneda:

```
# Experimento: Lanzamiento de una moneda
moneda = ['C', 'S'] # Cara y Sello

# Espacio muestral para 1 lanzamiento
omega_1 = list(itertools.product(moneda, repeat=1))
print(f"Espacio muestral 1 lanzamiento: {omega_1}")
print(f"Cardinalidad: {len(omega_1)}")

# Espacio muestral para 2 lanzamientos
omega_2 = list(itertools.product(moneda, repeat=2))
print(f"\nEspacio muestral 2 lanzamientos: {omega_2}")
print(f"Cardinalidad: {len(omega_2)}")

# Espacio muestral para 3 lanzamientos
omega_3 = list(itertools.product(moneda, repeat=3))
print(f"\nEspacio muestral 3 lanzamientos: {omega_3}")
print(f"Cardinalidad: {len(omega_3)}")
```

Espacio muestral 1 lanzamiento: [('C',), ('S',)]
Cardinalidad: 2

Espacio muestral 2 lanzamientos: [('C', 'C'), ('C', 'S'), ('S', 'C'), ('S', 'S')]
Cardinalidad: 4

Espacio muestral 3 lanzamientos: [('C', 'C', 'C'), ('C', 'C', 'S'), ('C', 'S', 'C'), ('C', 'S', 'S'), ('S', 'C', 'C'), ('S', 'C', 'S'), ('S', 'S', 'C'), ('S', 'S', 'S')]
Cardinalidad: 8

2. Genere el espacio muestral para el lanzamiento de un dado:

```
# Experimento: Lanzamiento de dados
dado = [1, 2, 3, 4, 5, 6]

# Un dado
omega_dado1 = list(itertools.product(dado, repeat=1))
print(f"1 dado - Cardinalidad: {len(omega_dado1)}")

# Dos dados
omega_dado2 = list(itertools.product(dado, repeat=2))
print(f"2 dados - Cardinalidad: {len(omega_dado2)}")

# Mostrar algunos resultados
print(f"\nPrimeros 10 resultados de 2 dados: {omega_dado2[:10]}")
```

1 dado - Cardinalidad: 6
2 dados - Cardinalidad: 36

Primeros 10 resultados de 2 dados: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (2, 1), (2, 2), (2, 3), (2, 4)]

3. Defina eventos y calcule sus cardinalidades:

```
# Definir eventos para 2 dados
# Evento A: La suma de los dados es 7
evento_A = [resultado for resultado in omega_dado2 if sum(resultado) == 7]
print(f"Evento A (suma = 7): {evento_A}")
print(f"Cardinalidad de A: {len(evento_A)}")

# Evento B: Ambos dados muestran numeros pares
evento_B = [resultado for resultado in omega_dado2 if all(x % 2 == 0 for x in resultado)]
print(f"Evento B (ambos pares): {evento_B}")
print(f"Cardinalidad de B: {len(evento_B)}")

# Evento C: Al menos un dado muestra 6
evento_C = [resultado for resultado in omega_dado2 if 6 in resultado]
print(f"Evento C (al menos un 6): {len(evento_C)} resultados")
```

Evento A (suma = 7): [(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)]
Cardinalidad de A: 6

Evento B (ambos pares): [(2, 2), (2, 4), (2, 6), (4, 2), (4, 4), (4, 6), (6, 2), (6, 4), (6, 6)]
Cardinalidad de B: 9

Evento C (al menos un 6): 11 resultados

ar Tarea 2: Calculo de Probabilidades - Enfoque Clasico

Caso de estudio: Calcular probabilidades usando el enfoque clasico (todos los resultados igualmente probables).

4. Implemente una funcion para calcular probabilidades:

```
def probabilidad_clasica(evento_favorable, espacio_muestral):
    """
    Calcula la probabilidad clasica de un evento.

    Parametros:
    - evento_favorable: lista de resultados favorables
    - espacio_muestral: lista de todos los resultados posibles

    Retorna:
    - Probabilidad del evento
    """
    return len(evento_favorable) / len(espacio_muestral)

# Ejemplo: Probabilidad de obtener suma 7 con 2 dados
P_A = probabilidad_clasica(evento_A, omega_dado2)
print(f"P(A) = P(suma = 7) = {len(evento_A)} / {len(omega_dado2)} = {P_A:.4f}")

# Probabilidad de ambos pares
P_B = probabilidad_clasica(evento_B, omega_dado2)
print(f"P(B) = P(ambos pares) = {P_B:.4f}")

# Probabilidad de al menos un 6
P_C = probabilidad_clasica(evento_C, omega_dado2)
print(f"P(C) = P(al menos un 6) = {P_C:.4f}")
```

P(A) = P(suma = 7) = 6/36 = 0.1667
P(B) = P(ambos pares) = 0.2500
P(C) = P(al menos un 6) = 0.3056

5. Calcule probabilidades de operaciones entre eventos:

```
# Interseccion de A y B
interseccion_AB = [r for r in evento_A if r in evento_B]
print(f"A ∩ B = {interseccion_AB}")
P_A_inter_B = len(interseccion_AB) / len(omega_dado2)
print(f"P(A ∩ B) = {P_A_inter_B:.4f}")

# Union de A y B
union_AB = list(set(evento_A + evento_B))
P_A_union_B = len(union_AB) / len(omega_dado2)
print(f"P(A ∪ B) = {P_A_union_B:.4f}")
```

```
# Verificar:  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ 
verificacion = P_A + P_B - P_A_inter_B
print(f"Verificacion:  $P(A) + P(B) - P(A \cap B) = \{verificacion:.4f\}$ ")
```

```
A ∩ B = []
P(A ∩ B) = 0.0000
```

```
P(A ∪ B) = 0.4167
Verificacion:  $P(A) + P(B) - P(A \cap B) = 0.4167$ 
```

ar Tarea 3: Enfoque Empirico - Simulacion

6. Simule el lanzamiento de dados:

```
def simular_dados(n_simulaciones):
    """
    Simula el lanzamiento de dos dados n veces.
    Retorna una lista con las sumas obtenidas.
    """
    resultados = []
    for _ in range(n_simulaciones):
        dado1 = np.random.randint(1, 7)
        dado2 = np.random.randint(1, 7)
        resultados.append((dado1, dado2))
    return resultados

# Realizar simulaciones con diferentes tamanos
tamanos = [100, 1000, 10000, 100000]

print("Simulacion de P(suma = 7):")
print(f"Probabilidad teorica: {P_A:.4f}\n")

for n in tamanos:
    simulacion = simular_dados(n)
    conteo_7 = sum(1 for r in simulacion if sum(r) == 7)
    prob_empirica = conteo_7 / n
    print(f"n = {n}:>6: P(suma=7) = {prob_empirica:.4f}, Error = {abs(prob_empirica - P_A):.4f}")
```

```
Simulacion de P(suma = 7):
Probabilidad teorica: 0.1667
```

```
n = 100: P(suma=7) = 0.1500, Error = 0.0167
n = 1000: P(suma=7) = 0.1550, Error = 0.0117
n = 10000: P(suma=7) = 0.1694, Error = 0.0027
n = 100000: P(suma=7) = 0.1653, Error = 0.0014
```

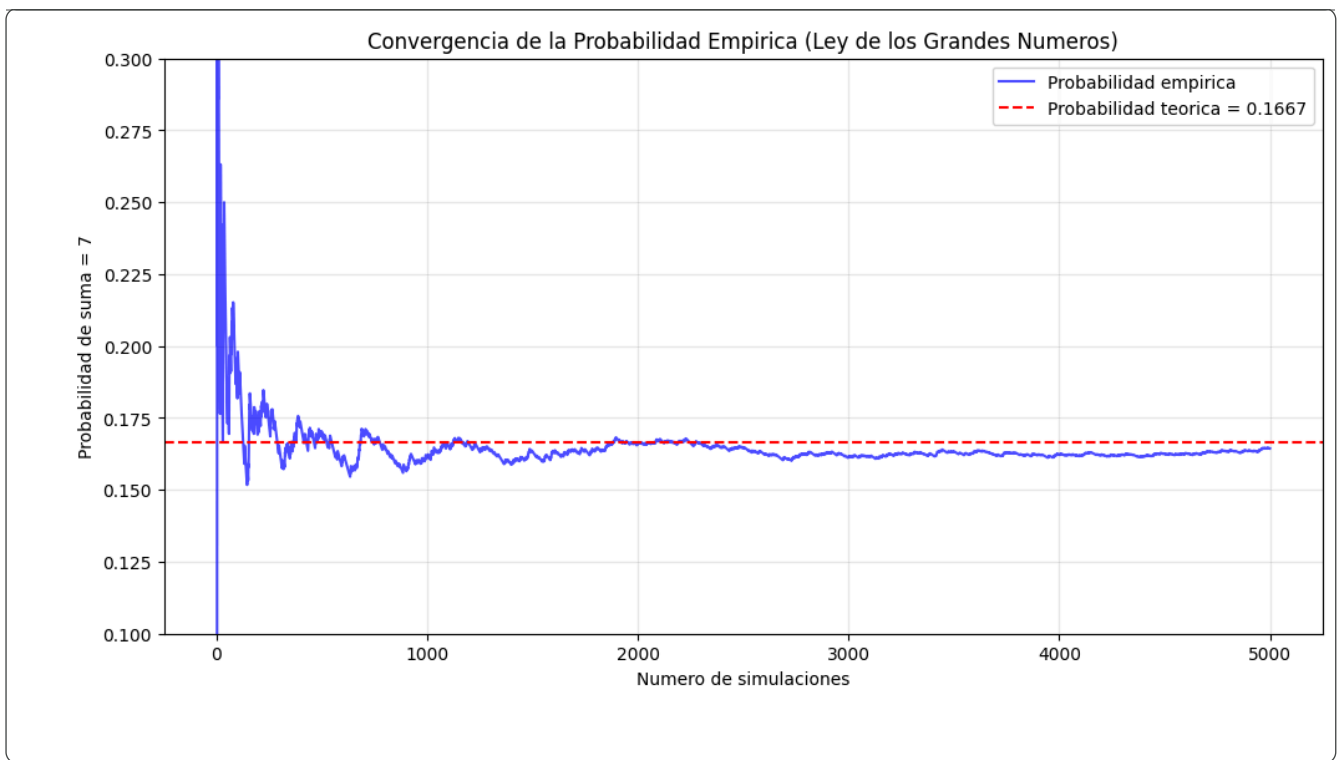
7. Visualice la convergencia:

```
import matplotlib.pyplot as plt

# Simulacion progresiva
n_max = 5000
simulacion = simular_dados(n_max)

# Calcular probabilidad acumulada
prob_acumulada = []
for i in range(1, n_max + 1):
    conteo = sum(1 for r in simulacion[:i] if sum(r) == 7)
    prob_acumulada.append(conteo / i)

# Graficar
plt.figure(figsize=(12, 6))
plt.plot(range(1, n_max + 1), prob_acumulada, 'b-', alpha=0.7, label='Probabilidad empirica')
plt.axhline(y=P_A, color='r', linestyle='--', label=f"Probabilidad teorica = {P_A:.4f}")
plt.xlabel('Numero de simulaciones')
plt.ylabel('Probabilidad de suma = 7')
plt.title('Convergencia de la Probabilidad Empirica (Ley de los Grandes Numeros)')
plt.legend()
plt.grid(alpha=0.3)
plt.ylim(0.1, 0.3)
plt.show()
```



Tarea 4: Probabilidad Condicional e Independencia

Caso de estudio: Analizar eventos condicionales e independencia entre eventos.

8. Calcule probabilidades condicionales:

```
# Probabilidad condicional:  $P(A|B) = P(A \cap B) / P(B)$ 
if P_B > 0:
    P_A_dado_B = P_A_inter_B / P_B
    print(f"P(A|B) = P(A ∩ B) / P(B) = {P_A_inter_B:.4f} / {P_B:.4f} = {P_A_dado_B:.4f}")

# Interpretacion: Dado que ambos dados son pares, cual es la probabilidad de que sumen 7?
print(f"Interpretacion: Si sabemos que ambos dados son pares,")
print(f"la probabilidad de que sumen 7 es {P_A_dado_B:.4f}")
```

$P(A|B) = P(A \cap B) / P(B) = 0.0000 / 0.2500 = 0.0000$

Interpretacion: Si sabemos que ambos dados son pares,
la probabilidad de que sumen 7 es 0.0000

9. Verifique independencia de eventos:

```
# Dos eventos son independientes si  $P(A \cap B) = P(A) * P(B)$ 
P_A_times_P_B = P_A * P_B
print(f"P(A) * P(B) = {P_A:.4f} * {P_B:.4f} = {P_A_times_P_B:.4f}")
print(f"P(A ∩ B) = {P_A_inter_B:.4f}")

if abs(P_A_inter_B - P_A_times_P_B) < 0.0001:
    print("\nLos eventos A y B son INDEPENDIENTES")
else:
    print("\nLos eventos A y B NO son independientes")
```

$P(A) * P(B) = 0.1667 * 0.2500 = 0.0417$
 $P(A \cap B) = 0.0000$

Los eventos A y B NO son independientes

